

1. **Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.**

```
# Hardcoded admin credentials
admin_username = "admin"
admin_password = "zomato123"

# User input
username = input("Enter Username: ")
password = input("Enter Password: ")

# Check credentials
if username == admin_username and password == admin_password:
    print("Access Granted")
else:
    print("Access Denied")
```

2. **Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the `resource`, otherwise `False`. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.
Hint: Use a dictionary to map roles to allowed resources.**
- ```
def check_access(user_role, resource):
```

```
 access = {
 "customer": ["order_history"],
 "restaurant_owner": ["menu_edit", "order_history"],
 "admin": ["menu_edit", "order_history", "admin_dashboard"]
 }
```

```
 if resource in access.get(user_role, []):
 return True
 return False
```

```
Test Cases
print(check_access("customer", "order_history")) # True
print(check_access("customer", "menu_edit")) # False
print(check_access("restaurant_owner", "menu_edit")) # True
print(check_access("admin", "admin_dashboard")) # True
```

- 3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.**

```
import random

correct_password = "paytm123"

password = input("Enter Password: ")

if password == correct_password:

 otp = random.randint(100000, 999999)

 print("Generated OTP:", otp)

 user_otp = int(input("Enter OTP: "))

 if user_otp == otp:
 print("Login Successful")
 else:
 print("Invalid OTP")

else:
 print("Incorrect Password")
```

- 4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.<br><br><em><strong>Constraint:</strong></em> Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.</em> from datetime import datetime**

```

users = [
 {"username": "Amit", "isPremium": True, "lastLogin": "2024-06-09"},
 {"username": "Riya", "isPremium": False, "lastLogin": "2024-06-08"},
 {"username": "Karan", "isPremium": True, "lastLogin": "2024-06-01"},
 {"username": "Neha", "isPremium": True, "lastLogin": "2024-05-30"}
]

today = datetime.strptime("2024-06-10", "%Y-%m-%d")

def premium_users(users):

 result = []

 for user in users:
 login_date = datetime.strptime(user["lastLogin"], "%Y-%m-%d")

 days = (today - login_date).days

 if user["isPremium"] and days <= 7:
 result.append(user)

 return result

print(premium_users(users))

```

**5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.**

| Mistake                                 | Prevention                                              |
|-----------------------------------------|---------------------------------------------------------|
| Weak passwords                          | Enforce strong password policy and minimum length.      |
| No Role-Based Access Control (RBAC)     | Give users only the permissions they need.              |
| Missing Two-Factor Authentication (2FA) | Enable OTP or authenticator app for sensitive accounts. |

| Mistake            | Prevention                                                        |
|--------------------|-------------------------------------------------------------------|
| Exposed Admin URLs | Protect admin pages with authentication and authorization checks. |